

UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA
CURSO DE GRADUAÇÃO EM SISTEMAS DE INFORMAÇÃO

Bruna Caetano e Renata Fonseca

SIMULADOR DE PROCESSADOR MIPS EM LINGUAGEM ASSEMBLY

Santa Maria, RS
2026

SUMÁRIO

1	OBJETIVOS.....	10
2	REVISÃO BIBLIOGRÁFICA.....	11
2.1	COMO UM PROCESSADOR MIPS FUNCIONA	11
2.2	FORMATO DAS INSTRUÇÕES MIPS.....	11
2.3	MEMÓRIA E REGISTRADORES.....	11
3	METODOLOGIA	12
3.1	AMBIENTE USADO	12
3.2	ESTRUTURA DE DADOS.....	12
3.3	PROCEDIMENTOS CRIADOS	12
3.4	LOOP PRINCIPAL	13
4	EXPERIMENTO	14
4.1	COMO O TESTE FOI FEITO	14
4.2	INSTRUÇÕES SUPORTADAS.....	14
4.3	EXEMPLO DE SAÍDA	15
5	DISCUSSÃO	16
5.1	ENDEREÇOS RELATIVOS	16
5.2	EXTENSÃO DE SINAL NO IMMEDIATE	16
5.3	PROTEÇÃO DO REGISTRADOR \$ZERO	16
6	CONCLUSÃO E PERSPECTIVAS.....	17
6.1	CONCLUSÃO.....	17
6.2	PERSPECTIVAS FUTURAS	17
	REFERÊNCIAS BIBLIOGRÁFICAS	18

1 OBJETIVOS

O objetivo deste trabalho é criar um programa em assembly MIPS que simule o funcionamento de um processador MIPS. O simulador deve ser capaz de carregar um programa em linguagem de máquina de um arquivo, executar cada instrução passo a passo e mostrar o que está acontecendo dentro do processador a cada ciclo.

2 REVISÃO BIBLIOGRÁFICA

2.1 COMO UM PROCESSADOR MIPS FUNCIONA

O processador MIPS possui conjunto de instruções reduzido, mas cada uma é simples e rápida de executar. Segundo Patteerson e Hennessy (2006), todo processador repete o mesmo ciclo para cada instrução: primeiro busca a instrução na memória (fetch), depois lê quais campos ela tem (decode) e por fim executa a operação (execute). O registrador PC sempre aponta para a próxima instrução, e o IR guarda a instrução que está sendo executada no momento.

2.2 FORMATO DAS INSTRUÇÕES MIPS

Cada instrução MIPS tem 32 bits e pode estar em um de três formatos. O tipo R é usado para operações entre registradores e tem os campos: opcode (6 bits), rs, rt, rd (5 bits cada), shamt (5 bits) e funct (6 bits). O tipo I é usado para valores imediatos, acesso à memória e desvios, com: opcode, rs, rt e um valor imediato de 16 bits. O tipo J é usado para saltos e tem apenas o opcode e um endereço de 26 bits. Essa organização em campos fixos facilita muito a implementação do decodificador.

2.3 MEMÓRIA E REGISTRADORES

A memória de um processador MIPS é dividida em três partes: o segmento de texto (onde ficam as instruções), o segmento de dados (onde ficam as variáveis do programa) e a pilha (usada para guardar informações temporárias durante chamadas de funções). O processador tem 32 registradores de 32 bits. O registrador \$0 (zero) é especial: ele é sempre zero e não pode ser alterado. O registrador \$29 (sp) aponta para o topo da pilha, iniciando em 0x7FFFEFFC.

3 METODOLOGIA

3.1 AMBIENTE USADO

O simulador foi desenvolvido no MARS 4.5, que é um programa gratuito para escrever e rodar código assembly MIPS. O MARS tem suporte a syscalls de arquivo (abrir, ler e fechar), que foram usadas para carregar os arquivos binários de entrada. O código todo fica em um único arquivo chamado simulador.asm..

3.2 ESTRUTURA DE DADOS

Na seção .data do programa foram declaradas: três áreas de 4096 bytes para os segmentos de texto, dados e pilha; um vetor de 128 bytes para os 32 registradores (4 bytes cada); e variáveis inteiras para o PC, o IR, o contador de ciclos e os oito campos da instrução atual (opcode, rs, rt, rd, shamt, funct, immediate e address).

3.3 PROCEDIMENTOS CRIADOS

- O simulador foi dividido em procedimentos com funções bem definidas:
- Inicializar: zera todos os registradores e coloca o PC em zero.
- Carregar_arquivo_binario: abre o arquivo, lê em blocos de 1024 bytes e salva no segmento correto. Durante a cópia, converte os bytes de little-endian para big-endian, montando cada palavra de 32 bits como $B3 \ll 24 \mid B2 \ll 16 \mid B1 \ll 8 \mid B0$.
- Verifica_endereco: checa se um endereço pertence ao intervalo válido de um segmento e retorna 1 (válido) ou 0 (inválido).
- Memoria_leitura e Memoria_escrita: leem e escrevem uma palavra de 32 bits na memória.
- Fetch: copia a instrução apontada pelo PC para o IR e incrementa o PC em 4.
- Decode: extrai os campos da instrução usando deslocamentos (srl) e máscaras (andi) e guarda nas variáveis globais.

- Executar: decide qual instrução rodar com base no opcode e no funct, e executa a operação.

3.4 LOOP PRINCIPAL

O loop principal faz o seguinte em cada ciclo: verifica se o PC passou do fim do arquivo; incrementa o contador de ciclos; imprime informações de depuração (ciclo, PC e IR); chama fetch, decode e executar; volta ao início. O simulador para quando encontra uma instrução syscall com $\$v0 = 1, 10$ ou 17 , ou quando o PC chega ao fim do arquivo.

4 EXPERIMENTO

4.1 COMO O TESTE FOI FEITO

Para testar o simulador, foram usados os arquivos ex-000-073.bin (instruções) e ex-000-073.dat (dados). Os arquivos foram colocados na pasta de trabalho do MARS e o simulador foi executado com Run -> Go, observando a saída no console e o painel Data Segment.

Durante os primeiros testes, o simulador carregava mas não executava nenhuma instrução, exibindo 'Tamanho do arquivo: 0 bytes'. Foram identificadas duas causas: os arquivos estavam em uma subpasta, mas o MARS busca sempre na pasta raiz; e a flag de controle de carga estava deixando o segundo arquivo sobrescrever o tamanho registrado do primeiro. Após corrigir o local dos arquivos e a lógica da flag primeiro_arquivo, o carregamento passou a funcionar corretamente, com mensagens de confirmação para cada arquivo.

4.2 INSTRUÇÕES SUPORTADAS

Tipo	Instrução	Opcode	Funct	O que faz
R	add	0	32	$\$rd = \$rs + \$rt$
R	sub	0	34	$\$rd = \$rs - \$rt$
R	and	0	36	$\$rd = \$rs \& \$rt$
R	or	0	37	$\$rd = \$rs \$rt$
R	sll	0	0	$\$rd = \$rs \ll \$rt$
R	srl	0	2	$\$rd = \$rs \gg \$rt$
R	syscall	0	12	encerra o programa
I	addi	8		$\$rt = \$rs + imm$
I	andi	12		$\$rt = \$rs \& imm$
I	ori	13		$\$rt = \$rs imm$
I	lw	35		$\$rt = mem[\$rs + off]$
I	sw	43		$mem[\$rs + off] = \rt
I	beq	4		se $\$rs == \rt : $PC += off * 4$
I	bne	5		se $\$rs \neq \rt : $PC += off * 4$
J	j	2		$PC = target * 4$

4.3 EXEMPLO DE SAÍDA

A saída no console do MARS durante a execução fica assim:

```
=== SIMULADOR MIPS INICIADO ===  
Carregando arquivo TEXT (instruções)...  
Arquivo TEXT carregado  
Tamanho do arquivo: XXX bytes  
--- CICLO 1 | PC: 0x00000000 | IR: 0x[instrucao]  
[ADDI] $8 = 0x[resultado]  
--- CICLO 2 | PC: 0x00000004 | IR: 0x[instrucao]
```

Cada linha mostra o número do ciclo, o valor atual do PC e o conteúdo do IR, seguido do efeito da instrução executada.

5 DISCUSSÃO

5.1 ENDEREÇOS RELATIVOS

Em vez de trabalhar com endereços de memória absolutos (como 0x00400000 para texto), o simulador usa posições relativas ao início de cada segmento. Isso simplifica muito o código e é suficiente para os programas de teste. Para programas mais complexos que usem endereços absolutos, seria necessário adicionar uma conversão de endereços.

5.2 EXTENSÃO DE SINAL NO IMMEDIATE

As instruções do tipo I têm um campo immediate de 16 bits que pode representar valores negativos. Porém, os registradores do MIPS têm 32 bits. Se o valor imediato fosse copiado diretamente para 32 bits sem tratamento, valores negativos (cujo bit mais significativo é 1) viriam como positivos grandes em vez de negativos. Para resolver isso, foi usada a seguinte sequência: extrair os 16 bits com `andi 0xFFFF`, depois aplicar `sll 16` para levar o bit de sinal para a posição 31, e por fim `sra 16` (shift aritmético) para propagar o sinal corretamente para os bits superiores.

5.3 PROTEÇÃO DO REGISTRADOR \$ZERO

Durante a implementação, foi necessário garantir que nenhuma instrução pudesse alterar o registrador \$0, que deve ser sempre zero. A solução adotada foi simples: antes de qualquer escrita em registrador, o simulador verifica se o índice de destino é zero. Se for, a escrita é simplesmente ignorada e a execução continua normalmente.

6 CONCLUSÃO E PERSPECTIVAS

6.1 CONCLUSÃO

O simulador de processador MIPS desenvolvido em assembly MIPS cumpriu os objetivos propostos. Ele implementa corretamente o ciclo de instrução, suporta 15 instruções dos tipos R, I e J, carrega arquivos binários de disco, simula três segmentos de memória e um banco de 32 registradores.

Simular o MIPS dentro do próprio MIPS obrigou a trabalhar diretamente com os campos de bit das instruções, os endereços de memória e a lógica de controle do processador, o que reforça muito o entendimento prático de como um computador funciona. Os testes com os arquivos fornecidos confirmaram o funcionamento correto do simulador.

6.2 PERSPECTIVAS FUTURAS

Para versões futuras, seria interessante: adicionar as instruções jal e jr para suportar chamadas de função; implementar mais syscalls (como imprimir e ler valores); aumentar o tamanho dos segmentos de memória; adicionar detecção de overflow; e implementar a tradução de endereços virtuais para permitir rodar programas compilados com endereços absolutos MIPS.

REFERÊNCIAS

PATTEERSON, D.; HENNESSY, J. **Organização e Projeto de Computadores**: A interface hardware/software. 5. ed. [S.l.]: Morgan Kaufmann / Elsevier, 2006.